The Hong Kong Polytechnic University
Department of Electrical and Electronic Engineering

EIE3360 Integrated Project Final Report

Edge-Deployed Object Detection and Autonomous Waste Sorting for Dual Mini-Games on Jetson Orin

Course	EIE3360 Integrated Project
Project Group	Group 24
Members	Wang Zihao (23101728D), Sun Yiran (23101201D)
Coordinators	Wang Zihao and Sun Yiran
Submission Date	20 April 2026

Table of Contents

1. Introduction 3
2. Background and Objectives 3
3. Methodology and Description 4
4. Milestone Achievement 6
5. Implementation 6
6. Results and Evaluation 11
7. Conclusion 14
References 15
Appendix A. Workload Distribution 16
Appendix B. Supplementary Materials 16

1. Introduction

This project developed an autonomous robot system for two course mini-games that required real-time object detection, material-based sorting, and reliable physical execution under strict arena rules. The final system combined onboard YOLOv11 inference on Jetson Orin with STM32-based low-level actuation so that perception, decision making, movement, grasping, and release could be completed in one embedded closed loop. The robot had to identify visually similar cans and boxes, decide the correct destination zone, and complete the required tasks without human intervention once the demonstration started.

The project was challenging because success did not depend on computer vision alone. It depended equally on dataset quality, confidence calibration, state-machine design, mechanical reliability, camera stability, battery condition, and repeated full-field tuning. In practice, many early failures were caused not by poor model accuracy but by wheel blockage, left-right drift, imperfect grasp geometry, and target-switching instability when the robot approached objects at close range.

The final system achieved three key outcomes. First, the robot platform was mechanically redesigned to eliminate a wheel-jamming problem at the structural level and to support both can-grasping and box-grasping using a simple but effective claw modification. Second, the perception pipeline was expanded from an early baseline to a final v9 dataset and a 10-class detector trained for deployment as a TensorRT engine. Third, the software evolved into a shared architecture with game-specific modes, target locking, beacon-guided delivery, and task-state control robust enough to obtain full marks in the Week 13 final demonstration.

2. Background and Objectives

Real-time embedded robotic systems require low-latency perception and immediate control response. In this project, Jetson Orin served as the onboard AI platform and performed detection directly on the robot, instead of relying on remote processing. This edge-side deployment was important because the course demonstration imposed a five-minute time limit for each game, a shared 15-minute demonstration slot, and strict no-touch rules after the robot started [2], [3]. Under these constraints, delaying inference or offloading vision externally would have weakened the closed-loop responsiveness of the system.

The object-detection backbone was built with the YOLO family, whose unified single-stage design is well known for strong real-time performance in detection tasks [4]. This made YOLOv11 a suitable practical choice for the project because the same inference output could be used immediately for target selection, alignment, and motion correction. However, the project required more than a detector with good offline metrics. The final system had to operate in a real arena with reflective can surfaces, visually similar objects, constrained camera perspective, and non-ideal mechanical behaviour.

According to the project requirement, Game One required the robot to collect three regular Coca-Cola mini cans, avoid the Coca-Cola No Sugar can, and place the valid cans upright into the left aluminium zone. Game Two required mixed-object sorting: Coca-Cola and Sprite cans had to be delivered to the left aluminium zone, while Vita Lemon Tea and Vita Soybean Drink boxes had to be delivered to the right beverage-carton zone [2]. The claw length also had to remain within the course limit, and the final result depended not only on correctness, but also on movement quality, stability, and penalty avoidance [2].

The objectives of this project were therefore: (1) to build a robust multi-class detector for the final arena objects and marker objects; (2) to integrate Jetson Orin, camera input, YOLOv11 inference, STM32 communication, differential-drive motion, and gripper control into one reliable embedded workflow; (3) to implement reusable task-state logic for both games in a shared software structure; (4) to redesign the robot mechanically so that it could reliably grasp both cans and boxes; and (5) to achieve stable real-robot performance in the final demonstration rather than only strong offline validation scores.

3. Methodology and Description

3.1 Hardware and software components

The final platform used the common course robot chassis with Jetson Orin, an onboard camera, STM32 motor-control subsystem, wheels, motors, and a front gripper. Model training and experiment iteration were performed on a laptop equipped with an NVIDIA GeForce RTX 5080 Laptop GPU (16 GB), while image collection for dataset expansion was performed with an iPhone 16 Pro Max. Roboflow was used for annotation, class management, and dataset export in YOLO format. Python scripts running on Jetson Orin handled inference, state logic, and communication with the robot-control library. The deployed model was exported as a TensorRT engine for onboard use.

Figure 1 shows the final perception–decision–control architecture. The camera streamed frames to Jetson Orin, which performed YOLOv11 inference and interpreted the current scene. A state machine then decided the next action, such as searching, aligning, moving forward, grabbing, releasing, retreating, or switching to the next target. Jetson Orin sent these commands to STM32, which handled wheel motion and servo actuation. After every motion segment, the robot re-observed the arena and updated the next decision. This repeated loop was essential because the project environment was too variable for one-shot movement execution.

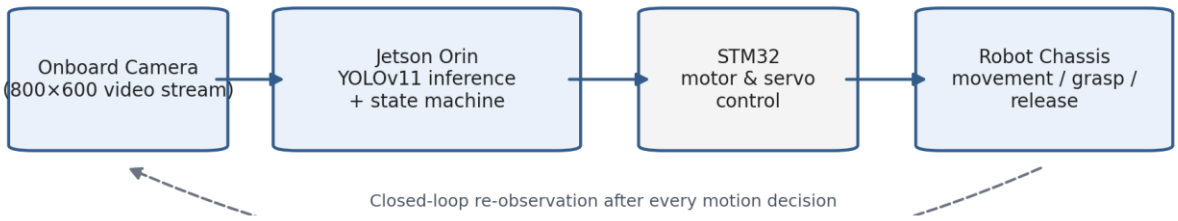


Figure 1. Final perception–decision–control architecture of the deployed robot system.

Category	Item	Purpose
Hardware	Jetson Orin + onboard camera	Onboard image acquisition, YOLOv11 inference, task-state decision making
Hardware	STM32 motor-control subsystem	Low-level wheel and servo control
Hardware	Course robot chassis, wheels, motors, claw	Physical movement, grasping, release
Hardware	Modified claw with 2 pencils + 4 rubber bands	Supports both can grasping and box grasping
Hardware	Training laptop (RTX 5080 Laptop GPU, 16 GB)	Model training and experiment iteration
Hardware	iPhone 16 Pro Max	Image and video collection for dataset expansion
Software	YOLOv11	Object detection training and inference
Software	Roboflow	Annotation, versioned dataset management, export
Software	Python + TensorRT engine deployment	Game logic and optimized onboard inference
Software	SSH / VNC tools	Remote deployment, monitoring, and debugging

Table 1. Hardware and software components used in the project.

3.2 Object detection process and dataset development

The object-detection pipeline followed a standard but heavily iterated engineering process. Images were collected in the actual laboratory environment, especially around the W402a floor and the same style of arena partitions used in testing. Labels were created in Roboflow, then exported as a YOLO dataset with

train/validation/test subsets. Training was performed with YOLOv11 at 640×640 resolution for 100 epochs, batch size 16, from a YOLO checkpoint rather than from scratch. The final model was then exported to TensorRT and deployed on the robot.

A critical methodological choice was to use one expanding dataset rather than multiple unrelated datasets. Instead of abandoning earlier data and restarting, the team iteratively refined the same project through multiple versions, each addressing a new failure mode or requirement. Early versions focused on baseline detection. Later versions added targeted top-view images, Coca-Cola No Sugar, multi-can backgrounds, marker-base cans, and more reference scenes, until the final v9 dataset supported both Game One and Game Two requirements. This versioned strategy made performance changes traceable and directly linked visual failure cases to dataset modification.

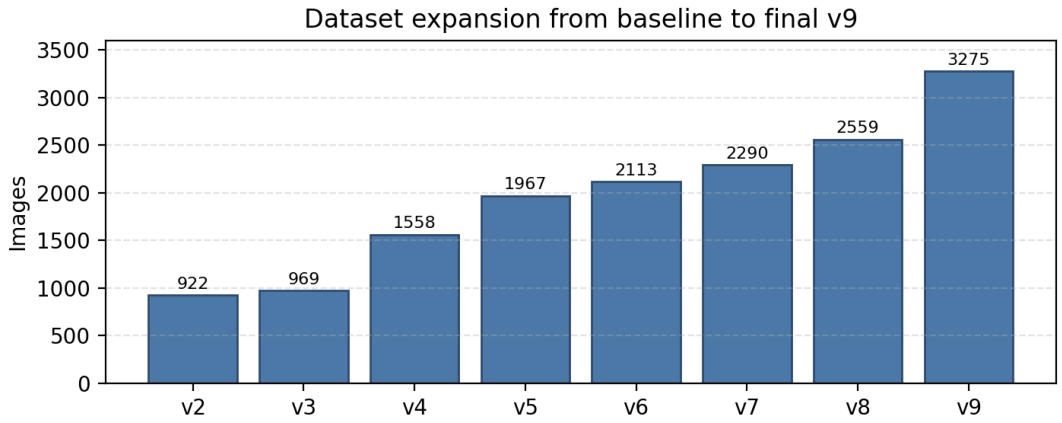


Figure 2. Growth of the dataset from the earlier verified baseline to the final v9 version.

3.3 Software–hardware integration and shared architecture

Another major methodological principle was to separate shared logic from game-specific logic. During development, Game One and Game Two were debugged in separate scripts for efficiency. However, the final design can be organized as a single program with selectable modes because both games use the same core modules: camera input, YOLO inference, confidence filtering, target alignment, movement primitives, claw control, retreat, and beacon-based delivery. The difference lies mainly in target-selection rules, destination mapping, and task-completion conditions.

4. Milestone Achievement

The project progressed through several clear milestones, each closing a real gap between an early proof-of-concept and a final demonstration system. The milestone structure became meaningful not because it matched a formal schedule on paper, but because each stage unlocked the next category of work: first a usable Game One baseline, then mechanical redesign, then final dataset completion, then Game Two stability, and finally demonstration readiness.

Time	Milestone	Significance and deliverables
Week 9	Stable Game One completion	Validated that the can-detection, exclusion, grasping, retreat, and beacon-guided placement pipeline could work end-to-end.
Week 10	Chassis drilling, motor repositioning, claw redesign, and re-tuning	Solved wheel blockage at the structural level; restored movement reliability; re-tuned thresholds and reproduced Game One on the redesigned robot.
Week 11	Final v9 dataset and final training completed	Locked the multi-class detector for both games and prepared the TensorRT deployment model.

Time	Milestone	Significance and deliverables
Week 12 (weekend)	Stable Game Two completion	Confirmed that the full mixed-object sorting pipeline could finish under arena conditions.
Week 13 (Monday)	Final demonstration	System achieved full marks in the official course demonstration.

Table 2. Major project milestones from stable Game One completion to final demonstration.

Major milestone challenges included wheel jamming after early assembly, unstable grasping caused by the original claw geometry, and detector confusion under top-view or multi-object scenes. These were overcome through chassis drilling and motor repositioning, claw redesign with pencil extensions, and iterative dataset expansion with repeated arena-side threshold tuning.

5. Implementation

Implementation required simultaneous progress in perception, control, and hardware adaptation. The final result was not produced by one large rewrite, but by repeated iterations that targeted concrete failure cases observed in real testing.

5.1 Dataset design, class definition, and final training setup

The final detection model was trained on a single Roboflow project that was expanded through nine versions. Earlier versions solved baseline feasibility, while later versions directly addressed errors observed during full-field testing. Version names such as “plus can top”, “Add zero sugar”, “4 can bg”, “add fanta schweppes”, and “add references” show that dataset development followed a problem-driven logic rather than random accumulation. By the final v9 stage, the dataset contained 3,275 images, split into 2,670 training images, 386 validation images, and 219 test images.

The final 10 classes were: coca cola can, coca cola can zero sugar, fanta can, green ball, pencil, red ball, schweppes can, sprite, vita lemon tea, and vita soybean drink. Not all of these were sorting targets. Some were deliberately introduced as auxiliary classes to suppress harmful false positives. For example, the “pencil” class was necessary because the externally bound pencils on the claw were visible in the camera view; without this class, the detector sometimes misinterpreted them as valid target objects. Similarly, Fanta and Schweppes were not to be sorted in Game Two, but still had to be detected because they acted as marker-base cans for the green and red football beacons, and otherwise could be confused with valid cans such as Sprite.

The class distribution in v9 was intentionally non-uniform because some classes were much harder than others. Coca-Cola can had the largest annotation count because it was central to both games and also the class most likely to be over-predicted if not modelled carefully. Coca-Cola zero sugar received focused expansion because misclassifying it in Game One would directly cost score. Sprite and Schweppes also received substantial attention because their appearance, especially from the top view, can become highly similar. The final class counts were 2,751 for coca cola can, 916 for coca cola can zero sugar, 200 for fanta can, 735 for green ball, 295 for pencil, 545 for red ball, 545 for schweppes can, 313 for sprite, 539 for vita lemon tea, and 370 for vita soybean drink.

Class	Role in system	Annotations
coca cola can	Primary sorting target in both games	2,751
coca cola can zero sugar	Exclusion class in Game One	916
fanta can	Left marker-base can	200
green ball	Left beacon	735
pencil	Auxiliary suppression class for claw-mounted pencils	295
red ball	Right beacon	545

Class	Role in system	Annotations
schweppes can	Right marker-base can	545
sprite	Aluminium target in Game Two	313
vita lemon tea	Beverage-carton target in Game Two	539
vita soybean drink	Beverage-carton target in Game Two	370

Table 3. Final v9 dataset classes and annotation counts.

The final training configuration was YOLOv11, image size 640×640, batch size 16, and 100 epochs. Training was performed on an RTX 5080 Laptop GPU and resumed from a YOLO checkpoint to accelerate convergence. After training, the selected model was exported as a TensorRT engine and used on the robot. This configuration balanced speed, reproducibility, and deployment practicality.

Confidence threshold selection had to be tuned separately for the two games. Game One used a threshold of 0.8 because it only needed reliable detection of regular Coca-Cola cans, Coca-Cola No Sugar, and the green beacon, and because the corresponding classes were relatively well represented in the final dataset. A higher threshold reduced the risk of false positives on irrelevant objects. Game Two used a lower threshold of 0.6 because the robot had to detect more categories, from more distances, and under more complex arrangements. If the threshold was set too low, background structures such as partitions or scene edges could be confused with real objects. If it was set too high, valid objects far from the robot were sometimes missed entirely. The final values were chosen through repeated arena-side testing rather than only from offline validation.

Setting	Final value
Model family	YOLOv11
Image size	640 × 640
Epochs	100
Batch size	16
Training device	NVIDIA GeForce RTX 5080 Laptop GPU (16 GB)
Training strategy	Resumed from YOLO checkpoint
Deployment format	TensorRT .engine
Final dataset version	v9
Total images / split	3,275 total; 2,670 train / 386 val / 219 test
Game One confidence	0.8
Game Two confidence	0.6

Table 4. Final training and deployment configuration.

5.2 Mechanical redesign and additional hardware improvements

The most important hardware contribution was a major redesign of the robot structure. In early assembly, the robot could not move normally because the inner side of the wheels was physically blocked by the chassis plates. A temporary attempt to enlarge the wheel-motor spacing reduced direct contact but made the wheels unstable. A second attempt raised the body height, but this introduced new problems: the lower plate still interfered with wheel motion, and the higher claw and camera geometry made can grasping less stable. The final solution attacked the root cause: new holes were drilled directly into the lower plate so that the motors could be repositioned further outward. Additional holes were also drilled to restore upper–lower plate fixation after the geometry changed. This redesign eliminated wheel jamming without permanently increasing robot height.

The claw was also modified in a highly practical manner. The original claw had anti-slip sleeves that made it difficult for mini cans to enter cleanly, so the sleeves were removed. To handle beverage cartons as well as cans, one pencil was attached to each outer side of the claw using rubber bands. This created a dual-purpose gripper: the inner original claw geometry remained suitable for near can-grasping, while the

outward pencil extension allowed wider and more distant contact for boxes. As a result, the same robot could handle two object families with different widths without requiring a complex re-fabricated gripper.

Even after these improvements, several mechanical limitations remained and had to be handled in software. The robot tended to drift left during nominal straight-line motion, motor speed control was coarse because the minimum effective value was still relatively high, the battery level could not be monitored directly and caused strange behaviour when low, and the camera angle could shift between sessions if the mount loosened. These limitations directly affected AI control performance because vision-based thresholds, centre alignment, and grasp distance all depend on stable geometry and predictable motion.

Mechanical issue	Effect on system	Implemented solution
Wheel jamming inside chassis	Robot could not move normally	Drilled new holes and repositioned motors outward; added new fixation holes to restore chassis stability
Left-right asymmetry / left drift	Reduced straight-line accuracy, especially for box grasping	Adjusted thresholds and repeatedly re-centred visually before grasp/release
Original claw too narrow; sleeves blocked can entry	Poor can fit and unstable capture	Removed sleeves to improve can entry
Original claw unsuitable for boxes	Could not grasp carton targets	Added 2 pencils with 4 rubber bands to widen effective contact geometry
Low battery and unobservable battery level	Unstable or abnormal behaviour during debugging	Repeated charging and test discipline; avoided judging performance from low-power runs
Loose camera angle	Threshold tuning became invalid across sessions	Tightened screws and rechecked camera alignment before tests

Table 5. Main mechanical limitations and the implemented solutions.

5.3 Game One implementation

Game One was implemented around the course rule that the robot should collect the regular Coca-Cola cans while excluding Coca-Cola No Sugar and deliver three valid cans to the left aluminium zone. The system first paused briefly to allow the scene to stabilize, then searched the rubbish area and selected the leftmost regular Coca-Cola can. This left-to-right selection rule matched the course requirement and simplified state progression.

After locking a valid can, the robot aligned the target using two vertical thresholds and then approached it until the bounding-box width exceeded a grasp threshold. The gripper closed, the robot retreated, and then turned toward the green-ball beacon on the left zone. The beacon was used as a reliable positional reference because direct placement based only on dead-reckoning would have been too sensitive to drift and floor variation.

A notable design improvement was the use of three different drop columns for Game One instead of repeatedly driving toward the same central drop point. Without this strategy, previously placed cans or the Fanta/green-ball marker base could be knocked over by later deliveries. Therefore, the robot distributed its three deliveries across left, middle, and right drop routes near the beacon. This simple route planning reduced interference between successive placements and improved the chance that all cans remained upright inside the zone.

Another essential improvement was target locking. During close approach, a neighbouring Coca-Cola No Sugar can could temporarily be misread as a regular Coca-Cola can, especially when only part of the object remained visible. Without locking, the robot could oscillate between targets and fail to complete the grasp. The final logic therefore kept following the currently selected object, even if nearby detections fluctuated, until the approach–grasp phase completed or the target was clearly lost.

5.4 Game Two implementation

Game Two required a more advanced state machine because the robot had to sort two aluminium cans and two beverage cartons into different destination zones. The system still selected the leftmost valid target among the remaining objects, but once the target class was determined, it was immediately mapped to an object family and destination side: coca cola can and sprite were mapped to the left aluminium zone, while vita lemon tea and vita soybean drink were mapped to the right beverage-carton zone.

Separate grasp configurations were implemented for cans and boxes. Can-grasping used the inner original claw and a nearer grasp geometry, while box-grasping relied more on the pencil-extended outer contact width and therefore used a different width threshold and alignment strategy. This distinction was necessary because the same motion settings could not reliably handle both object families. In testing, box grasping was the stricter problem because even small left-right error could cause one pencil to strike and topple the box before capture.

Beacon-guided delivery was extended to both sides in Game Two. The green ball acted as the left-zone beacon and the red ball as the right-zone beacon. After grasping a target, the robot turned toward the correct beacon, approached it, and then used a two-route drop strategy on each side so that previously delivered items would not be disturbed. The task ended after two objects had been delivered to each side, matching the course requirement.

A second form of target locking was added in Game Two to cope with class switching at very close range. When the camera observed mostly the can top surface, classes such as coca cola can, sprite, and schweppes can could alternate frame-to-frame because their visible features were reduced. The final logic therefore maintained commitment to the current target during grasp execution even if the instantaneous class label changed, preventing the entire state machine from collapsing at the most sensitive moment.

5.5 Modifications beyond the original baseline

Compared with the original baseline AIR script, the final implementation introduced major modifications. The original script mainly demonstrated a simple two-stage sequence: detect a Coca-Cola can, move toward it, then search for a green ball. The final system instead used complete state machines for both games, target-family mapping, carried-target context, beacon-specific delivery, route management, retreat phases, and finished-state logic. This shifted the program from a demonstration sample to a task-complete application.

Additional software features that improved performance included: (1) separate confidence settings for Game One and Game Two; (2) target locking and proxy tracking to prevent target switching during close approach; (3) distinct can and box grab configurations; (4) multi-column drop routing to reduce self-interference; (5) periodic command heartbeat and explicit stop states to improve control robustness; and (6) consistent re-observation after every motion segment instead of long uncontrolled movement blocks. These features did not merely add code complexity; they solved failure modes that were repeatedly observed during real robot testing.

Modification	Contribution to final performance
Single-sample script → dual game state machines	Moved from a simple sample sequence to full task-complete logic for Game One and Game Two
Target locking	Prevented the robot from switching targets during close approach when neighbouring objects or partial views changed the detection result
Proxy tracking	Allowed the robot to keep pursuing the same object even if only part of the locked target remained visible
Separate can/box grab configurations	Supported two object families with different grasp distances and widths
Beacon-guided delivery	Used green/red beacons as robust drop-zone references instead of relying on dead-reckoning only
Multi-column / multi-route placement	Reduced collisions with marker bases and previously placed objects

Modification	Contribution to final performance
Game-specific confidence thresholds	Balanced recall and false-positive control differently for the two tasks

Table 6. Key modifications introduced beyond the original baseline AIR script.

5.6 Testing procedures

Testing was performed in three layers. First, offline training results were checked using the YOLO results curves, precision, recall, mAP, and confusion matrix. Second, bench-side tests were used to verify camera connection, inference speed, basic movement, gripper opening/closing, and serial communication before entering the full arena. Third, repeated full-field tests were performed under actual arena conditions, including real partitions, floor surface, object placement patterns, and beacon markers.

Arena-side testing focused on practical task success rather than only static detection quality. The team repeatedly tuned alignment thresholds, grasp width thresholds, retreat times, turn durations, drop-route offsets, and confidence settings. Tests were also used to evaluate failure cases such as knocking over a box, colliding with a marker base, missing a distant target, oscillating left and right near a close object, and behaviour degradation under low battery. In this sense, testing was not a final verification step only; it was the main engine of system development.

5.7 Strategies and techniques for learning to learn (L2L)

The project provided a strong Learning to Learn (L2L) experience because almost every major improvement came from diagnosing an unfamiliar failure, designing a controlled fix, and validating it through iteration. The most important new skills developed were dataset design for real object detection, systematic relabelling and class refinement in Roboflow, YOLOv11 training and deployment, TensorRT export, Jetson-side debugging through SSH/VNC, and robot-specific state-machine design under hardware constraints.

Learning did not happen through one source alone. Teacher slides provided the initial workflow and course expectations, but the final solution required much deeper self-exploration. The team repeatedly performed trial-and-error tuning, compared script behaviour before and after each modification, analysed confusing frames, expanded the dataset for specific failure cases, and discussed alternatives together during debugging. In practice, the most useful pattern was not passive reading but evidence-based iteration: observe a failure, isolate its cause, change one factor, test again, and keep only the modifications that improved real behaviour.

This project also changed the team's understanding of what it means to "train an AI model". At the beginning, it was easy to think that a high mAP score would be sufficient. The real project showed that deployment success depends on the entire pipeline: camera angle, object distance, target selection logic, battery state, wheel symmetry, gripper geometry, background bias, and safe retreat strategy. The most valuable L2L outcome was therefore the growth of an engineering mindset: learning became a cycle of observation, diagnosis, redesign, and verification rather than simple parameter tuning.

Team learning was also highly collaborative. Because the group only had two members, almost all major design decisions were discussed and tested jointly. Both members participated across image collection, relabelling, model refinement, Jetson deployment, robot tuning, code revision, and report preparation. This close collaboration reduced hand-off loss and allowed faster debugging because visual, mechanical, and algorithmic problems could be examined together rather than in isolation.

5.8 Team collaboration

Because the group had only two members, collaboration was highly integrated rather than strictly partitioned. Almost all critical decisions—including dataset expansion, class definition, threshold selection, drilling and chassis redesign, claw modification, motion tuning, and debugging of state logic—were

discussed and verified jointly. In practice, both members worked across image collection, labelling, training refinement, Jetson deployment, robot tuning, code revision, and report preparation. This co-construction model made communication efficient and ensured that visual, mechanical, and algorithmic issues were solved together rather than sequentially.

For reporting purposes, the final workload is recorded as 50%–50%. This reflects the actual working pattern more accurately than assigning one member to only software and the other to only hardware. The project progressed through repeated paired testing, mutual checking of observed failures, and immediate redesign after each test session.

6. Results and Evaluation

6.1 Offline model performance

The final v9 detector achieved strong offline performance. From the uploaded training record, the best precision reached 0.9875, the best recall reached 0.9901, the best mAP50 reached 0.9911, and the best mAP50-95 reached 0.9813 during the 100-epoch training run. The final-epoch values also remained high, indicating stable convergence rather than a one-epoch spike. The training curves show smooth reduction of training and validation losses together with saturation of the main evaluation metrics, which supports the decision to deploy the trained model.

The normalized confusion matrix further showed strong diagonal dominance across all 10 classes. In other words, when the correct object was visible under the labelled conditions, the detector separated classes very well. At the same time, the matrix and real-robot tests also confirmed that background-related false positives remained a realistic issue, especially for classes that were frequent or visually dominant. This matched the team's own observation that the large number of coca cola can samples could bias the model toward predicting Coca-Cola on irrelevant objects or ambiguous shapes. The response was not to trust offline metrics blindly, but to combine them with threshold tuning and arena-side validation.

Metric	Value
Best precision	0.9875
Best recall	0.9901
Best mAP50	0.9911
Best mAP50-95	0.9813
Final-epoch precision	0.9866
Final-epoch recall	0.9886
Final-epoch mAP50	0.9906
Final-epoch mAP50-95	0.9797

Table 7. Key offline validation metrics from the final v9 training run.

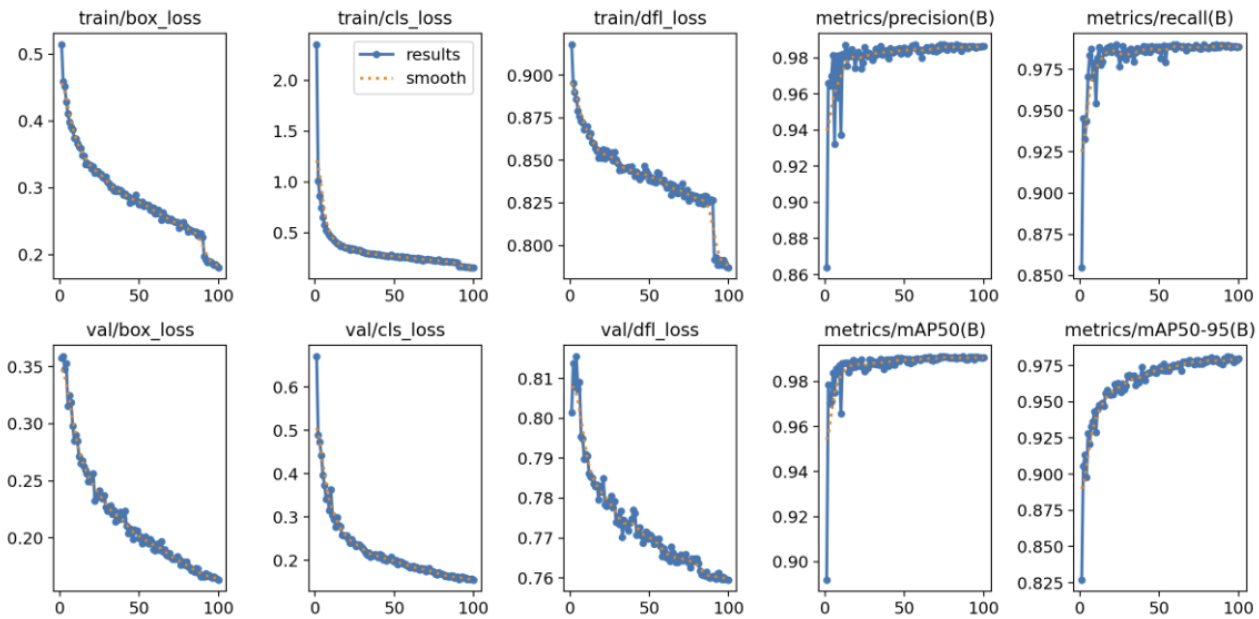


Figure 3. Training curves of the final model (losses, precision, recall, mAP50, and mAP50-95).

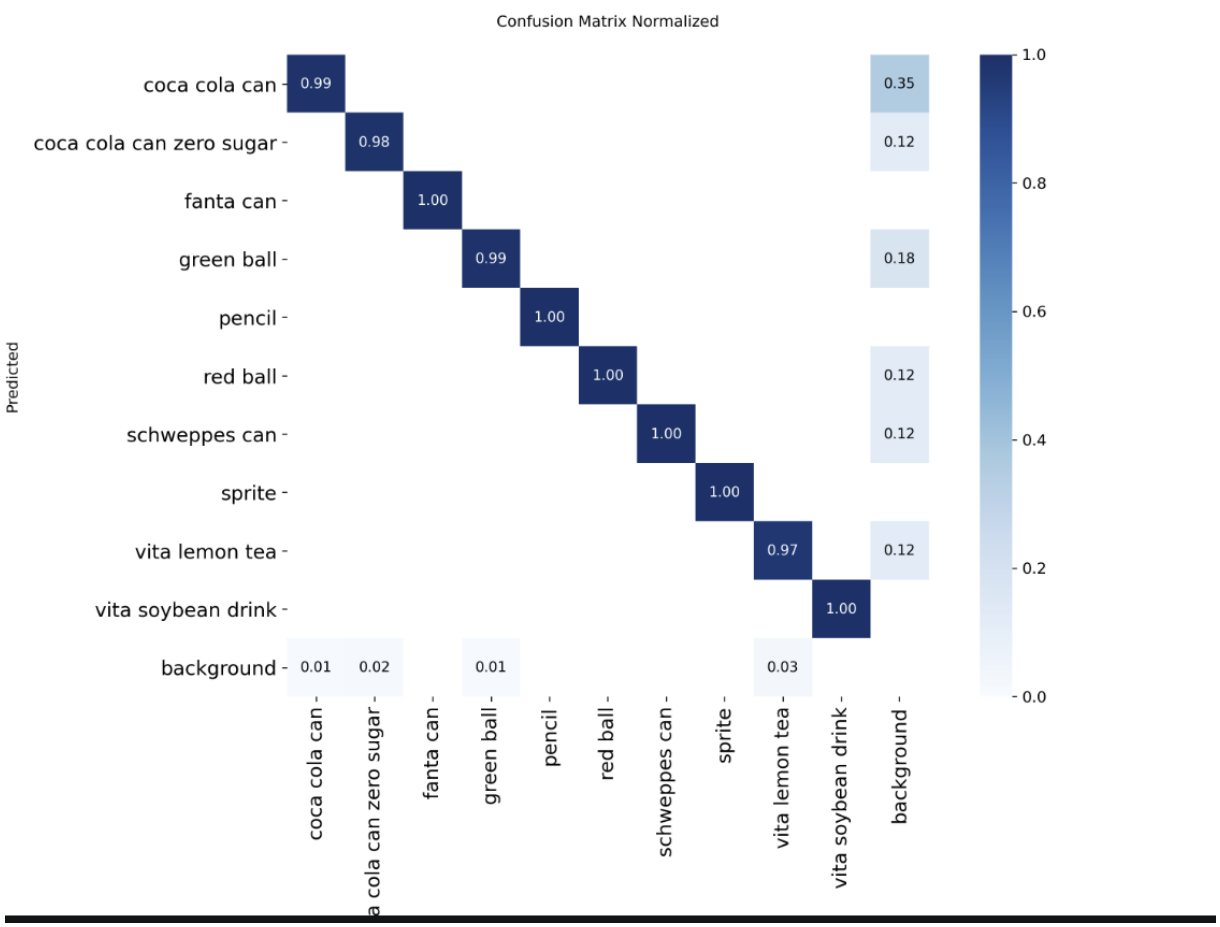


Figure 4. Normalized confusion matrix of the final 10-class detector.

6.2 Real-robot outcomes and practical evaluation

In full-field tests, Game One became stable earlier than Game Two. After the major implementation and tuning cycle, Game One could usually be completed successfully within two minutes, and across

approximately ten full tests only about one run showed a clear failure such as dropping a can or knocking it during close oscillation. Importantly, the system did not wrongly collect Coca-Cola No Sugar during these tests, which indicates that the detector and exclusion logic were strong enough for the course scoring requirement.

Game Two was more difficult because correct completion required both class recognition and object-family-specific grasping. The average behaviour was that around one out of five full tests still showed a problem, usually related to box grasp precision or momentary merged-object perception, for example when two nearby items were seen as one object. Nevertheless, after the v9 dataset, mechanical adjustments, and repeated threshold tuning, the team reached stable Game Two completion by the end of Week 12. In the Week 13 final demonstration, the system achieved full marks, as explicitly confirmed on site by the TA.

The real-robot inference speed on Jetson Orin was around 20–40 FPS. The VNC view looked heavily delayed, but the robot itself continued to run normally, so the delay was mainly in remote visualization rather than in onboard decision making. This distinction is important: the effective control loop should be evaluated based on robot behaviour, not based only on the smoothness of the remote desktop window.

Overall, the project met its objectives. The detector was extended to the full final object set, the onboard perception–control loop was integrated successfully, both games were implemented with reusable shared logic, the mechanical structure was redesigned to support reliable motion and grasping, and the final system achieved the desired demonstration performance. The main remaining weaknesses were not fundamental feasibility problems, but quality-of-control issues such as threshold-based oscillation near close targets, remaining drift sensitivity, and dependence on stable battery and camera setup.

Evaluation item	Observed outcome	Main limiting factor
Game One	Usually completed successfully within < 2 minutes; about 1 clear failure in ~10 full tests	Close-range left-right oscillation near a can due to enlarged bounding-box jitter
Game Two	Stable completion achieved by Week 12 weekend; about 1 problem in ~5 full tests during development	Box grasp precision, merged-object perception, and class switching near can tops
Final demo	Full marks in Week 13 final demonstration	TA explicitly confirmed the result on site
Onboard inference	Approximately 20–40 FPS on Jetson Orin	VNC view was delayed, but robot-side execution remained normal

Table 8. Real-robot test outcomes and observed limiting factors.

6.3 Areas for improvement

The main improvement area is motion correction. The current approach is threshold-triggered and behaves like a bang-bang controller: if the detected centre crosses the left or right boundary, the robot stops and applies a short fixed-duration correction. This works, but it causes visible “clicking” or oscillation near the target because small close-range visual changes produce large centre shifts. A more advanced proportional strategy, where left/right wheel speeds vary linearly with the signed deviation from the image midline, would likely produce smoother and more accurate approach behaviour.

Additional improvement areas include stronger battery monitoring, a more rigid camera mount, finer motor-speed control, more balanced data for frequently over-predicted classes, and automatic logging of test videos and failure events. These improvements are realistic extensions of the current system rather than entirely new research directions.

7. Conclusion

This project successfully transformed an early object-detection baseline into a full embedded robotic sorting system for two different course games. The final report has shown that success depended on the combination of three factors: a high-quality iteratively expanded dataset, a practical and well-tuned software state machine, and substantial mechanical redesign of the robot itself. The work therefore represents not only a computer-vision application, but also a complete embedded AI control system.

Three contributions were especially significant. First, the structural drilling and motor repositioning removed the root cause of wheel blockage and made the platform usable. Second, a very low-cost claw redesign using pencils and rubber bands enabled the same robot to handle both cans and boxes effectively. Third, the final detector achieved strong accuracy across all required classes and supported reliable real-time deployment in the arena. Together, these contributions were sufficient for the system to obtain full marks in the final demonstration.

The project also made clear that further improvement is still possible. If one to two more weeks were available, the most valuable software upgrade would be to replace the current threshold-triggered left/right correction with a continuous linear correction strategy based on the signed deviation between the target centre and the image midline. In principle, this proportional approach would reduce close-range oscillation and could also compensate more naturally for the robot's left-drift tendency, provided that motor speed control is sufficiently precise.

Beyond the course gameplay, the current hardware and software could be extended into a more general autonomous sorting platform. Potential improvements include continuous steering control, battery monitoring, a more rigid camera mount, odometry or IMU-assisted localization, stronger gripper feedback, richer event logging, and a more balanced dataset collected under multiple backgrounds and lighting conditions. Responsible AI practice in this context means recognizing dataset bias early, documenting it honestly, and reducing deployment risk through targeted data collection, conservative thresholds, and repeated real-world validation.

References

[1] EIE3360 Project Requirement v1.4, Department of Electrical and Electronic Engineering, The Hong Kong Polytechnic University, updated 21 Feb. 2026.

[2] EIE3360 Integrated Project – Project Report Guidelines, Department of Electrical and Electronic Engineering, The Hong Kong Polytechnic University, 2026.

[3] W. Zihao and S. Yiran, “EIE3360 Integrated Project Proposal: Jetson Nano-Based Real-Time Object Detection and Robotic Waste Sorting for Game One and Game Two,” Group 24, 9 Mar. 2026.

[4] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 779–788.

[5] NVIDIA, “Jetson Orin Nano Developer Kit Getting Started Guide,” NVIDIA Developer, accessed Apr. 2026.

[6] EIE3360 Integrated Project – Report for Lab 1–3 (Submit One Report): Report Guidelines, Department of Electrical and Electronic Engineering, The Hong Kong Polytechnic University, 2026.

[7] Roboflow, “Roboflow Universe / annotation and dataset management platform,” accessed Apr. 2026.

Appendix A. Workload Distribution

The final workload distribution is recorded as 50% for each member. This balanced distribution reflects the actual workflow of the project, in which both members jointly participated in image collection, dataset refinement, model training, Jetson deployment, robot tuning, software debugging, and report preparation.

Member	Percentage	Main work completed
Wang Zihao (23101728D)	50%	Image collection, dataset expansion, labelling, model training/refinement, Jetson deployment, robot tuning, software debugging, report drafting
Sun Yiran (23101201D)	50%	Image collection, dataset expansion, labelling, model training/refinement, Jetson deployment, robot tuning, software debugging, report drafting

Appendix B. Supplementary Materials

The appendix below provides supplementary evidence used during development and reporting. These materials support the implementation narrative but are excluded from the 15-page main-content limit.

B1. Final dataset evidence: The final Roboflow dataset reached v9 with 3,275 images and a train/validation/test split of 2,670 / 386 / 219. The dataset was expanded through multiple versions to address failure cases such as confusing can-top views, the addition of Coca-Cola No Sugar, four-object background arrangements, marker-base objects, and more reference scenes.

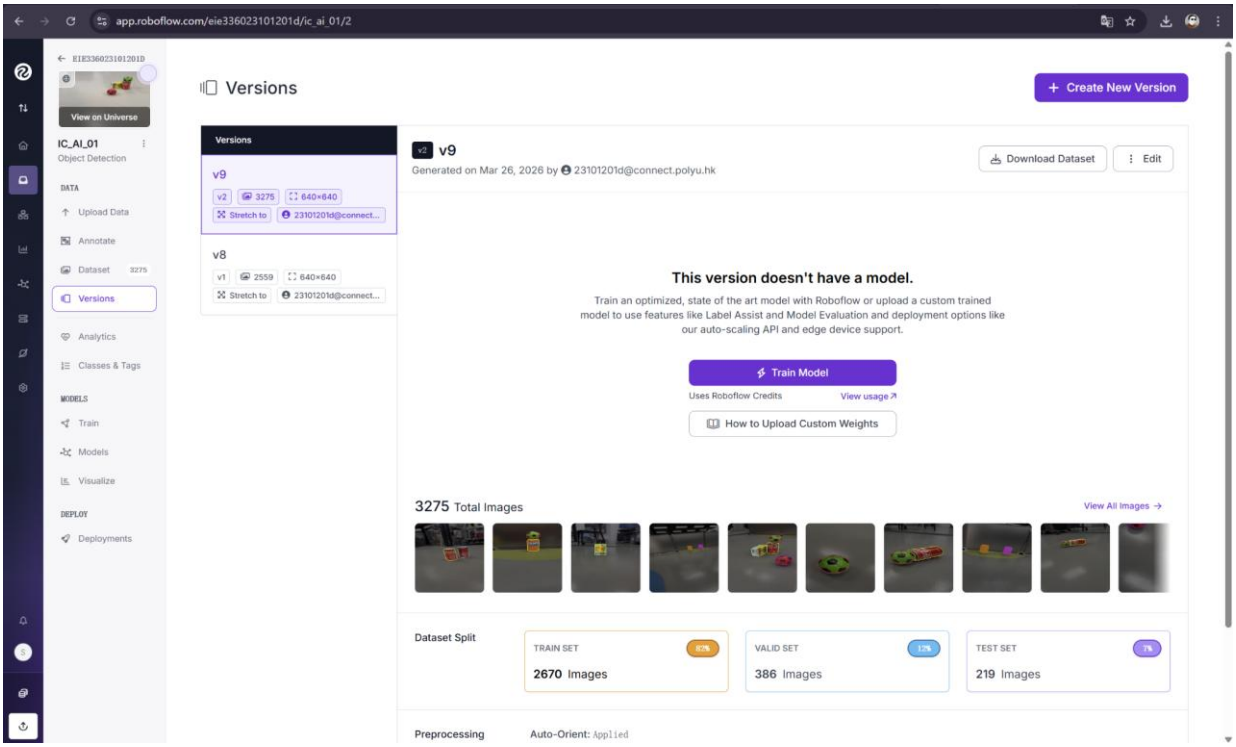


Figure A1. Roboflow v9 dataset summary showing total image count and split.

B2. Class list evidence: The final 10 classes were managed within the same Roboflow project so that one detector could support both games and also suppress important false positives.

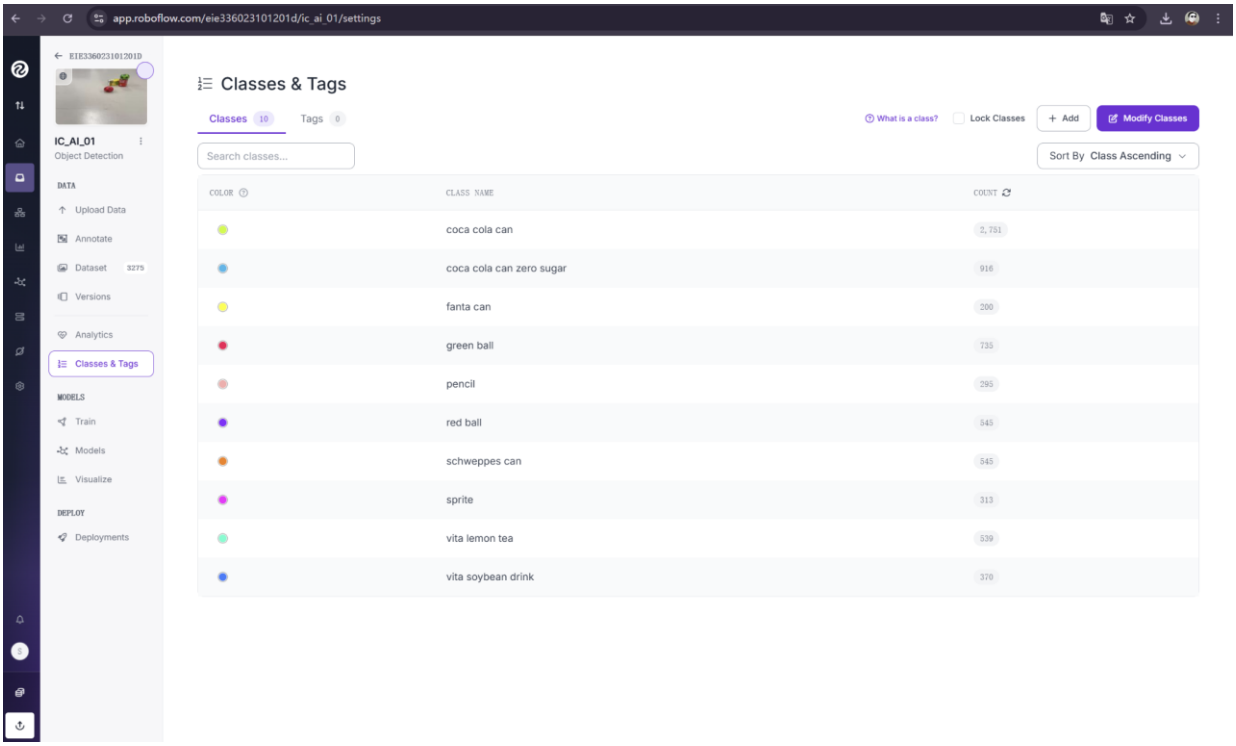
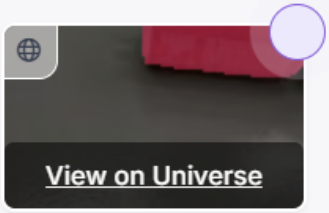


Figure A2. Final class list and annotation counts in the Roboflow project.

B3. Iterative version history: the project did not jump directly to the final result. It progressed through multiple dataset versions, each corresponding to concrete observations from testing.

ZIHAO



Lab3360

Object Detection

DATA

- Upload Data
- Annotate
- Dataset 2290
- Versions
- Analytics
- Classes & Tags

MODELS

- Train
- Models
- Visualize

DEPLOY

- Deployments

Versions

Versions

Stretch to

23101728d@connec...

4 can bg

v5 | 1967 | 640×640

Stretch to

23101728d@connec...

Add zero sugar

v4 | 1558 | 640×640

Stretch to

23101728d@connec...

plus can top

v3 | 969 | 640×640

Stretch to

23101728d@connec...

2026-02-26 9:35pm

v2 | 922 | 640×640

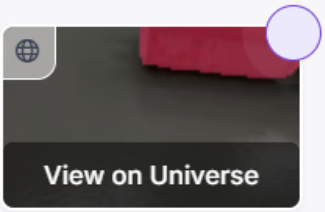
Stretch to | Notes

23101728d@connec...

Roboflow Instant 1 [Eval]

v1 | 922

ZIHAO



Lab3360

Object Detection

DATA

- Upload Data
- Annotate
- Dataset 2290
- Versions
- Analytics
- Classes & Tags

MODELS

- Train
- Models
- Visualize

DEPLOY

- Deployments

Versions

Versions

add references

v7 | 2290 | 640×640 | Stretch to | 23101728d@connec...

add fanta schwppes

v6 | 2113 | 640×640 | Stretch to | 23101728d@connec...

4 can bg

v5 | 1967 | 640×640 | Stretch to | 23101728d@connec...

Add zero sugar

v4 | 1558 | 640×640 | Stretch to | 23101728d@connec...

plus can top

v3 | 969 | 640×640 | Stretch to | 23101728d@connec...

Figure A3. Intermediate dataset versions used to refine the final detector.

B4. Example baseline script and final task-state scripts: during development, a simpler baseline AIR script was extended into separate game-specific state machines, which can also be integrated under one selectable mode-based framework.

B5. Because the final demonstration was performed live and the system achieved full marks, no formal supplementary video file was recorded. The report therefore uses training figures, dataset records, code-logic evidence, and documented real-robot outcomes as the main supporting materials.